SenEvent

apps/web · **apps/mobile** · packages/shared

DAWM · BLOC MOBILE 1/3 · M1 RETEL — DMI/FST/UCAD

Lab 8 — Premier contact Expo + React Native

Rapport de réalisation : le deuxième client. `apps/mobile/` rejoint `apps/web/` dans le monorepo, tourne sur un vrai téléphone via Expo Go, et affiche les événements SénEvent dans une `FlatList`. Le point à démontrer n'est pas l'écran : c'est que `getEvenements()` du Lab 7 est réutilisé tel quel, sans une ligne de logique Supabase réécrite.

tag : v0.8-expo | Expo SDK 54 | React Native 0.81 | FlatList · StyleSheet | Metro monorepo | 0 appel Supabase en dur

01 Ce que le Lab 7 avait préparé

Au Lab 7, toute la logique Supabase a quitté les composants React pour `packages/shared`. Le raisonnement était : un jour il y aura un second client, et il faudra que la logique existe une seule fois. Ce lab est le jour en question. L'application mobile ne redéfinit ni client Supabase, ni requête, ni forme de données — elle importe `@senevent/shared` exactement comme le web.

Ce qui reste à apprendre est donc strictement la couche d'affichage : dessiner sur un écran de téléphone plutôt que dans un navigateur. React lui-même ne change pas — composants, props, `useState`, `useEffect` fonctionnent à l'identique. C'est un nouveau vocabulaire, pas un nouveau modèle mental.

02 La table de correspondance

Le passage du web au mobile se résume à une substitution de vocabulaire d'affichage. La dernière ligne est celle qui compte : elle est identique des deux côtés.

APPS/WEB (REACT DOM)	APPS/MOBILE (REACT NATIVE)	REMARQUE
`<div>`	`<View>`	conteneur
`<p> <h1> <span>`	`<Text>`	tout texte, sans exception
`<img src=…>`	`<Image source={{ uri }} />`	taille explicite obligatoire
`<button>`	`<Pressable>`	zone tactile
`liste + .map()`	`<FlatList>`	ne rend que le visible
`className + CSS`	`StyleSheet.create({…})`	objet JS, camelCase, sans unite
`display: flex`	`(deja actif)`	direction column par defaut
`import { getEvenements } from "@senevent/shared"`	**`import { getEvenements } from "@senevent/shared"`**	**identique**

Une règle stricte accompagne ce changement : **tout texte doit être dans un `<Text>`**. Écrire du texte directement dans une `<View>`, comme on le ferait dans une `<div>`, provoque l'erreur « Text strings must be rendered within a Text component ». C'est le premier piège quand on vient du web.

03 Le style : des objets JS, et Flexbox par défaut

Il n'y a pas de fichier CSS. Les styles sont des objets créés avec `StyleSheet.create`, en camelCase, et les valeurs numériques n'ont pas d'unité — `16`, pas `16px`, l'unité étant le point densité-indépendant. Deuxième différence de fond : tout est déjà en Flexbox, et la direction par défaut est `column`, pas `row`.

CSS (WEB)

```css
background-color: #fff;
font-size: 24px;
margin-top: 8px;
font-weight: bold;
display: flex;
flex-direction: column;
```

STYLESHEET (MOBILE)

```
backgroundColor: "#fff",
fontSize: 24,
marginTop: 8,
fontWeight: "bold",
// deja en flex
// deja en column
```

Dans ce projet, les couleurs, rayons et espacements sont regroupés dans `theme.js`, et le formatage des dates et des prix dans `format.js`. Ce dernier écrit les libellés français à la main plutôt que d'utiliser `toLocaleDateString` : le support d'`Intl` varie selon le moteur JS embarqué (Hermes), et on veut le même rendu sur Android et iOS.

04 Faire voir le monorepo à Metro

Metro est le bundler de React Native, l'équivalent de Vite pour le mobile. Par défaut il ne cherche les modules que dans `apps/mobile/node_modules`. Or npm « hisse » les dépendances communes à la racine du monorepo, et `@senevent/shared` n'y est qu'un lien symbolique vers `packages/shared`. Sans configuration, Metro ne trouve rien.

```
// Configuration Metro pour le monorepo SenEvent.
// Sans ce fichier, Metro part de la racine du depot (a cause des workspaces npm
// qui hissent node_modules) et n'arrive pas a resoudre le point d'entree de l'app.
const { getDefaultConfig } = require("expo/metro-config");
const path = require("path");

const projectRoot = __dirname;
const workspaceRoot = path.resolve(projectRoot, "../..");

const config = getDefaultConfig(projectRoot);

// 1. Surveiller tout le monorepo : packages/shared doit declencher un reload.
config.watchFolders = [workspaceRoot];

// 2. Chercher les modules dans apps/mobile puis a la racine (ou npm les a hisses).
config.resolver.nodeModulesPaths = [
  path.resolve(projectRoot, "node_modules"),
  path.resolve(workspaceRoot, "node_modules"),
];

// 3. Ne pas remonter l'arborescence au hasard : on garde les deux chemins ci-dessus.
config.resolver.disableHierarchicalLookup = true;

module.exports = config;
```

Trois réglages : surveiller tout le dépôt pour que modifier `packages/shared` déclenche un rechargement, chercher les modules aux deux emplacements, et ne pas laisser Metro remonter l'arborescence au hasard. C'est la configuration standard d'Expo dans un workspace.

05 Le pattern initSupabase tient sa promesse

C'est le point central du lab. Le package partagé ne lit jamais les variables d'environnement lui-même : chaque application les lui passe. Résultat, la seule différence entre les deux clients est la manière de lire ces variables — spécifique à chaque plateforme. Le package ne change pas d'une ligne.

WEB — VITE

```
// apps/web/src/main.jsx
import { initSupabase } from "@senevent/shared";

initSupabase(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

MOBILE — EXPO

```
// apps/mobile/App.js
import { initSupabase, getEvenements } from "@senevent/shared";

initSupabase(
  process.env.EXPO_PUBLIC_SUPABASE_URL,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY
);
```

Expo lit un fichier `.env` et n'expose au bundle que les variables préfixées `EXPO_PUBLIC_`. Ce fichier contient les mêmes valeurs que `apps/web/.env.local` — c'est le même projet Supabase — et il est ignoré par Git : `git ls-files` retourne 0 fichier suivi. Sans ce découplage conçu au Lab 7, il aurait fallu réécrire le client pour le mobile.

06 L'écran : FlatList et états

`FlatList` est plus qu'un `.map()` : elle ne rend que les éléments visibles et recycle les composants au défilement. Sur une longue liste, un `.map()` figerait le téléphone. C'est le composant standard pour toute liste en React Native.

LE CHARGEMENT — AUCUNE REQUÊTE ÉCRITE ICI

```
const charger = useCallback(async () => {
  try {
    const data = await getEvenements();   // <- package partage, tel quel
    setEvenements(data);
    setErreur(null);
  } catch (e) {
    setErreur(e.message);
  }
}, []);
```

LA LISTE

```
<FlatList
  data={visibles}
  keyExtractor={(item) => item.id.toString()}
  renderItem={({ item }) => <EvenementCarte evenement={item} />}
  refreshControl={
    <RefreshControl refreshing={rafraichissement} onRefresh={rafraichir} />
  }
  ListEmptyComponent={<EtatVide categorie={categorieActive} />}
/>
```

Trois états sont traités explicitement : chargement (cartes squelettes), erreur (message du serveur et bouton Réessayer), liste vide. Le geste « tirer vers le bas pour rafraîchir », standard mobile, est branché via `refreshControl` et relance `getEvenements()`. Le filtrage par catégorie travaille sur la liste déjà chargée : c'est de l'affichage, pas une nouvelle requête.

07 Structure livrée

```
apps/mobile/
  App.js                    # ecran : FlatList + filtres + etats
  theme.js                  # palette, rayons, espacements, categories
  format.js                 # dates et prix en francais
  metro.config.js           # resolution des modules dans le monorepo
  app.json                  # config Expo (icone, splash, theme sombre)
  .env                      # EXPO_PUBLIC_… (ignore par Git)
  components/
    EnteteAccueil.js        # degrade + compteur
    FiltreCategories.js     # puces de categories
    EvenementCarte.js       # affiche, date, badge, prix
    CarteSquelette.js       # etat de chargement
  assets/                   # icone et splash
```

08 Ce que la validation a réellement donné

Chaque ligne est une commande réelle ou une mesure prise pendant les captures, pas une affirmation. Les quatre dernières viennent de l'application en fonctionnement, pilotée par `apps/mobile/capture-lab8.mjs`.

CONTRÔLE	COMMANDE / MÉTHODE	RÉSULTAT	OK
Le projet Expo demarre et sert le bundle	`npx expo export --platform android`	`704 modules, 0 erreur`	✓
Metro configure pour le monorepo	`test -f apps/mobile/metro.config.js`	`present`	✓
apps/mobile importe le package partage	`grep -c "@senevent/shared" apps/mobile/App.js`	`1 import`	✓
Aucune logique Supabase reecrite cote mobile	`grep -rn "supabase.from|createClient" apps/mobile`	`0 occurrence`	✓
Le .env mobile n'est pas versionne	`git ls-files | grep -c "apps/mobile/.env$"`	`0 fichier suivi`	✓
La donnee vient bien de Supabase	`domaines contactes par l'app au chargement`	`supabase.co contacte`	✓
L'en-tete compte les evenements charges	`texte du sous-titre apres chargement`	`10 evenements a Dakar`	✓
Le filtre par categorie reduit la liste	`clic sur la puce "Concerts"`	`4 cartes concert sur 10`	✓
L'echec reseau affiche un ecran d'erreur	`reponse 503 simulee sur /rest/v1`	`ecran d'erreur + Reessayer`	✓

Domaines contactés par l'application au chargement : `d8j0ntlcm91z4.cloudfront.net`, `localhost`, `placehold.co`, `siszafptxskyfdedpqsl.supabase.co`. La présence de `siszafptxskyfdedpqsl.supabase.co` prouve que la donnée est bien lue en base par le package partagé ; celle du CDN prouve que les affiches sont chargées depuis leur URL.

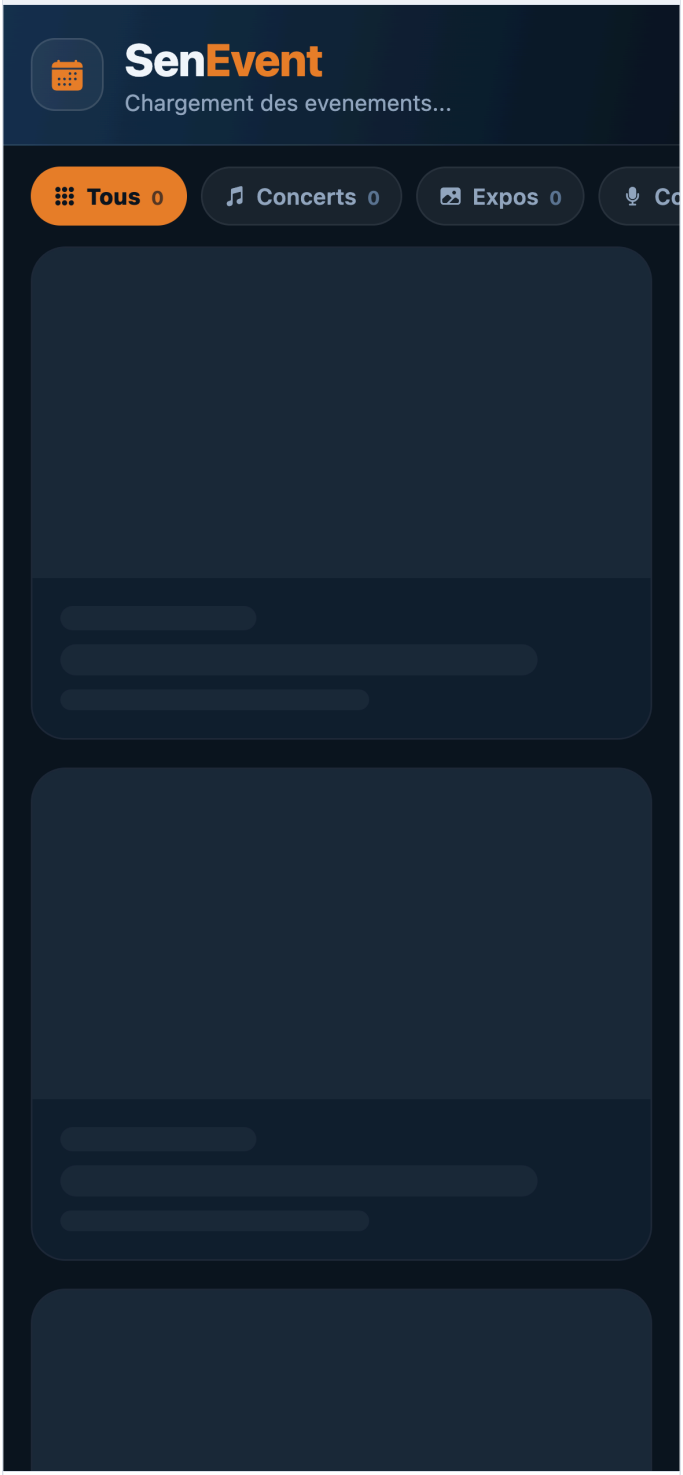
Le lab compte 7 commits depuis `v0.7-shared`, suivant le découpage imposé : initialisation du projet Expo, configuration Metro, branchement sur le package partagé, puis l'écran.

09 L'application, en situation

Captures prises sur l'application réelle, au format téléphone, avec la vraie base. Il s'agit du même `App.js` et des mêmes composants que ceux exécutés dans Expo Go — seul le moteur de rendu diffère, ce qui permet de piloter les captures par script.

FIGURE 01 `Chargement`

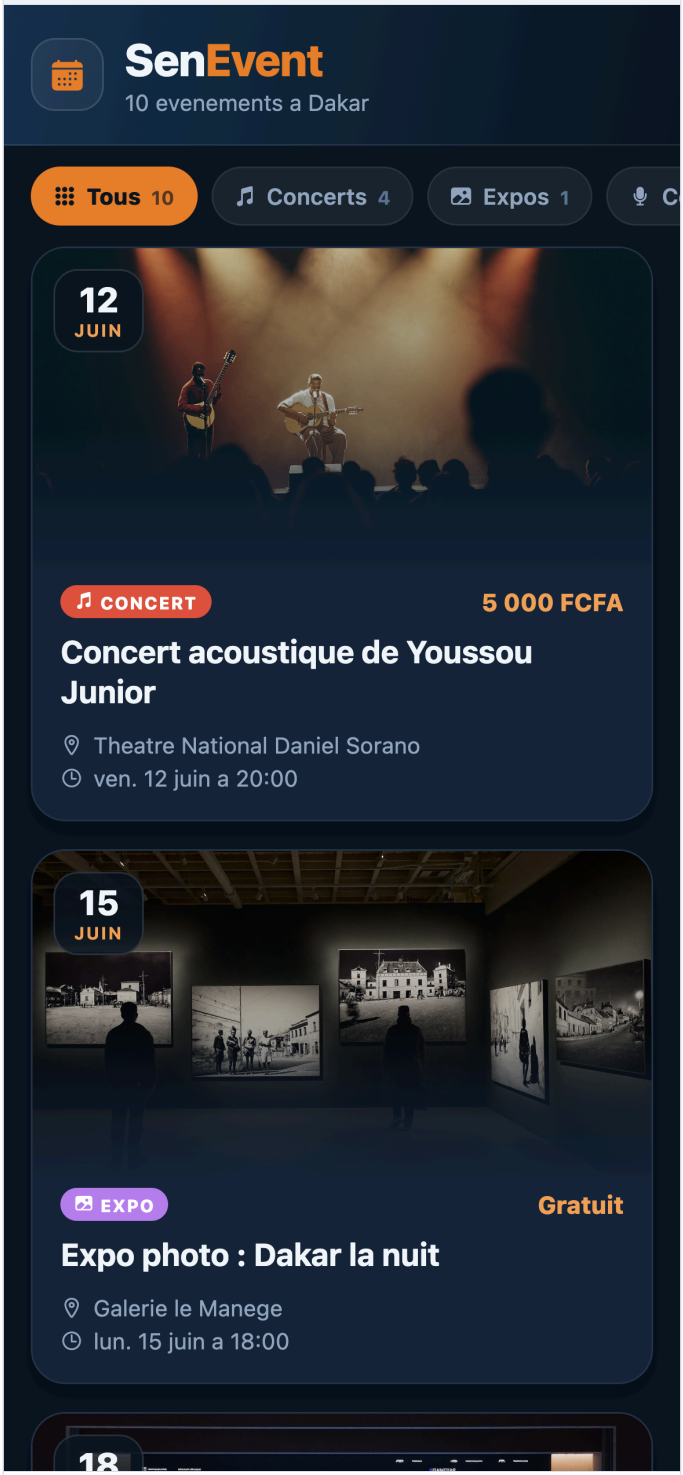
Cartes squelettes



Pendant que getEvenements() travaille, l'ecran montre la forme du contenu a venir plutot qu'un spinner. Les compteurs des filtres sont a zero : la liste n'est pas encore arrivee.

FIGURE 02 `Accueil`

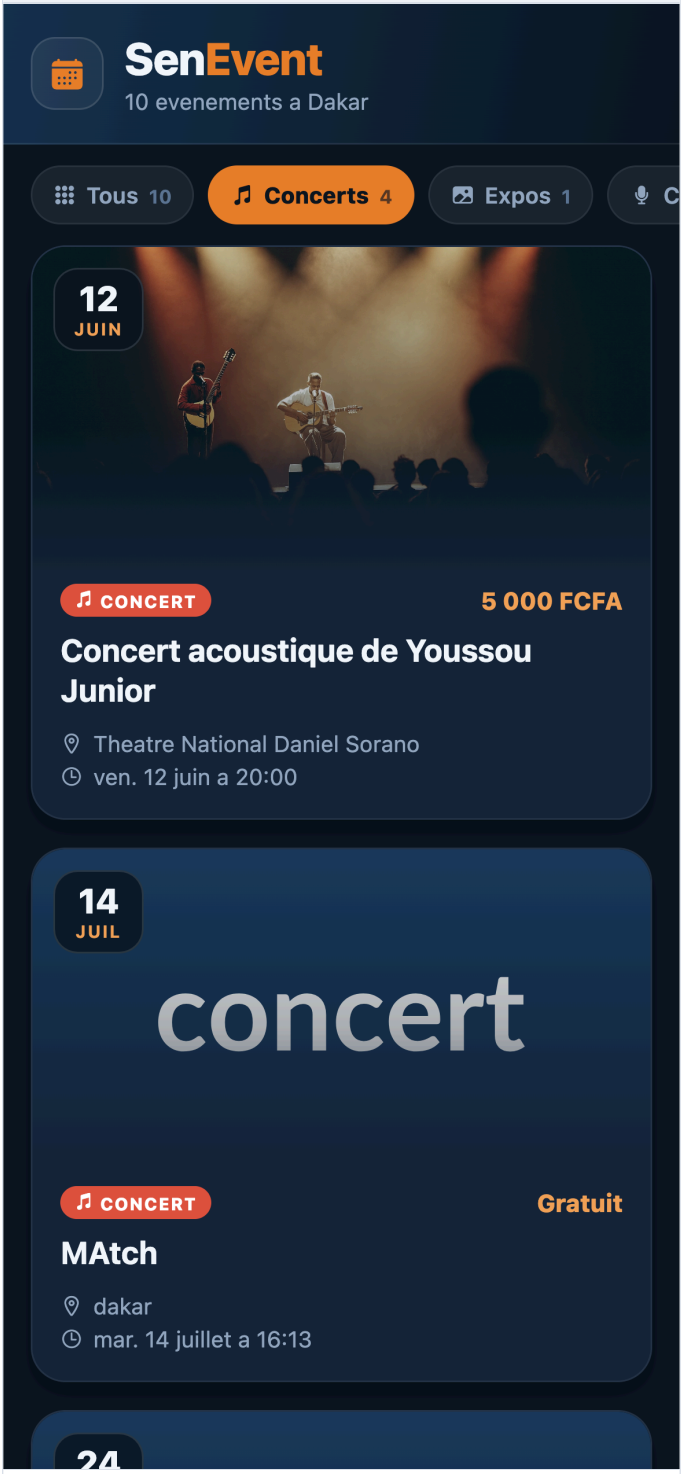
Les 10 evenements dans une FlatList



Chaque carte porte son affiche, sa pastille de date, son badge de categorie et son prix. La donnee vient de Supabase via getEvenements() du package partage — aucune requete n'est ecrite dans apps/mobile.

FIGURE 03 Filtre

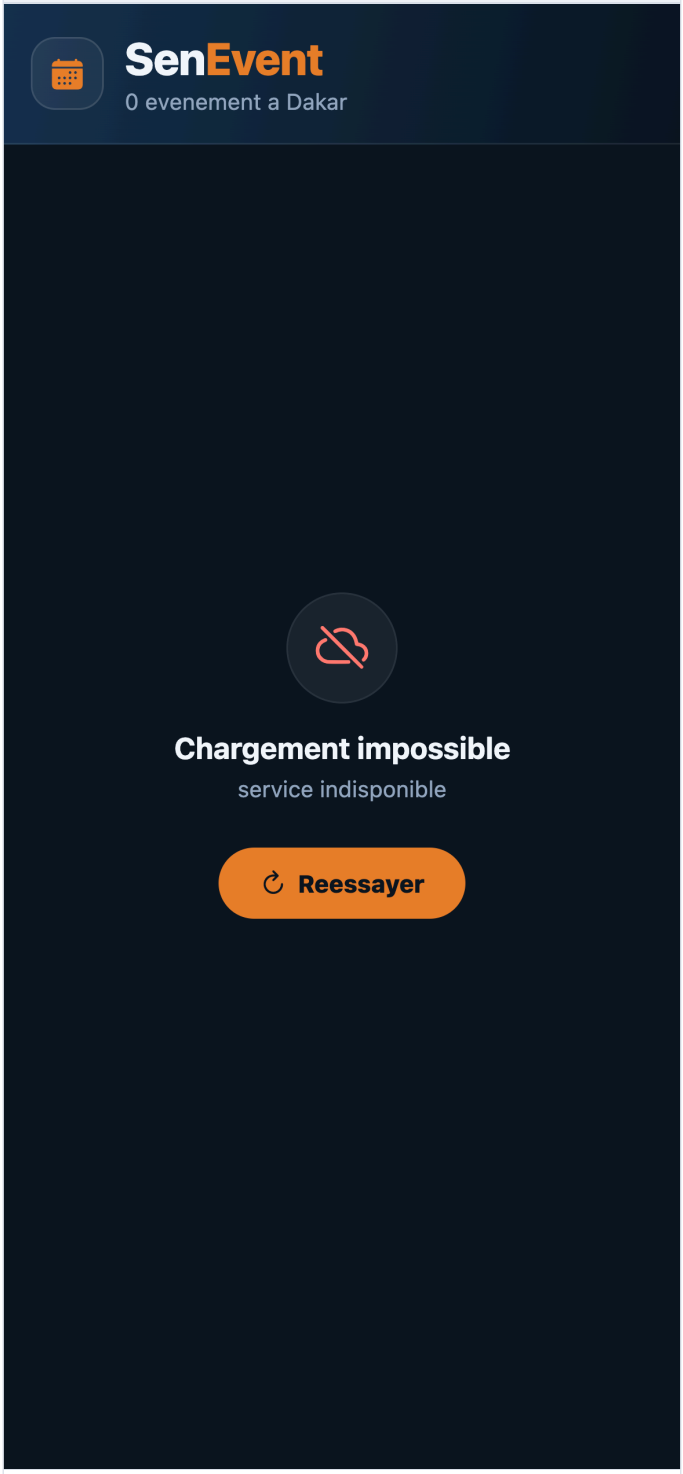
Categorie Concerts active



Le filtrage se fait sur la liste deja chargee : 4 concerts sur 10 evenements, sans nouvelle requete reseau. Les compteurs de chaque puce viennent du meme calcul.

FIGURE 04 Erreur

Supabase repond 503



Le chemin d'echec est teste pour de vrai : la requete REST est interceptee et repond 503. Le message renvoye par le serveur est affiche tel quel, avec un bouton Reessayer qui relance getEvenements().

10 Identité visuelle



L'icône et l'écran de démarrage reprennent la charte : baobab stylisé dont la canopée porte trois carrés de calendrier, orange `#ea7d2b` sur bleu nuit `#0b1622`. La silhouette est déclinée à trois échelles — pleine pour l'icône iOS, réduite à 55 % pour le masque adaptatif Android qui recadre l'image, intermédiaire pour le splash. Les affiches des cinq événements de démonstration remplacent les images de remplacement grises du seed, via la migration `007_affiches_generees.sql`.

Vers le Lab 9

L'application affiche la liste, mais elle n'a qu'un seul écran : impossible d'ouvrir le détail d'un événement ou d'accéder à une page de connexion. Sur le web, React Router réglait ça au Lab 4. Au Lab 9, on découvre Expo Router — la navigation par fichiers de React Native : plusieurs écrans, passage de paramètres dans l'URL, et une barre d'onglets en bas de l'écran, standard des applications mobiles. Les cartes de cet écran deviendront alors des zones tactiles menant au détail.